Geoscientific Machine Learning

Pankaj K Mishra

Table of Contents

1 Geoscientific Machine Learning

Welcome to **Geoscientific Machine Learning**.

If you're reading this, you probably already have some familiarity with geoscience. You're used to thinking in terms of physical processes, models, scales, and uncertainty. When it comes to computation, though, you may have mostly worked through existing tools: running models, adjusting parameters, or modifying scripts written by someone else. That's a common place to be, and it works, until you want to change the model itself or test an idea that doesn't fit into the existing workflow.

You might also be here because you keep hearing about AI and machine learning and you're not sure what to make of it. Is it actually useful for your research, or is it just hype? Will it help you work faster, or will it send you down a rabbit hole? Can it help you do things you thought were out of reach before, or is it mostly a waste of time?

Before you decide, it's worth trying it in a way that respects how geoscience works. That's what this book is about.

By **scientific machine learning**, we mean instead of training a model only on data and hoping it behaves, you guide the learning with what you already know: physical laws, conservation principles, symmetries, and numerical structure. The goal is not just to fit observations, but to produce models that behave sensibly when you change conditions or run them for longer times. By **geoscientific machine learning**, we mean applying scientific machine learning to geoscience problems. That means combining geoscience domain knowledge with learning methods so the models respect physics, scale relationships, and observational constraints.

If you don't yet have a favorite programming language, or you're still deciding what's worth learning, here's the important part: the exact language matters less than having *one*. You need a way to talk to computers clearly, so they can help you turn ideas into experiments and hypotheses into something you can test.

In this book we'll choose **Julia**. If you spend time with it, you'll see why—especially once you start doing more demanding computation. We'll begin slowly and keep things practical, using Julia to express ideas you already know. Machine learning comes later where it connects naturally geoscience practices.

If you're new to programming, expect some friction at first (we'll try to minimise that). That's normal. The payoff is that you gain more control over your models and more confidence in the results they produce.

This book is under construction

Some sections are incomplete. If there's something you'd like to see included, open an issue and let me know what you're working on.

2 Getting started with Julia

We recommend starting with **VS Code** and adding **Julia** via juliaup. Keep it lean: install VS Code, install Julia, point VS Code to Julia, set your depot location, and add a few key extensions.

2.1 Quick setup (Windows, macOS, Linux)

1. Install VS Code from code.visualstudio.com.
2. Install Julia with juliaup (recommended everywhere):

   - **Windows (PowerShell)**

     ```
     winget install julia -s msstore
     juliaup add release
     ```

   - **macOS (Terminal, zsh/bash)**

     ```
     curl -fsSL https://install.julialang.org | sh
     juliaup add release
     ```

   - **Linux (Terminal, bash/zsh)**

     ```
     curl -fsSL https://install.julialang.org | sh
     juliaup add release
     ```

3. Verify Julia is on PATH:

   ```
   julia --version
   ```

2.2 Set Julia depot location (where .julia lives)

Pick a location (e.g., `D:\JuliaDepot` on Windows, `/opt/julia-depot` or `~/julia-depot` on macOS/Linux) and set `JULIA_DEPOT_PATH` before launching Julia/VS Code.

- **Windows (PowerShell, per-shell)**

```
$env:JULIA_DEPOT_PATH = "D:\JuliaDepot"
julia --version
```

- **Windows (persist system/user env var)**

```
setx JULIA_DEPOT_PATH "D:\JuliaDepot"
```

- **macOS/Linux (zsh/bash)**

```
export JULIA_DEPOT_PATH="$HOME/julia-depot"
julia --version
```

- To keep multiple depots (rare), separate with `;` on Windows or `:` on macOS/Linux.

2.3 Point VS Code to Julia

If auto-detect fails, set **Settings → Julia: Executable Path**:

- **Windows**: `%LOCALAPPDATA%\Programs\Julia-*\bin\julia.exe` or `%USERPROFILE%\.juliaup\bin\julia.exe`
- **macOS**: `/Applications/Julia-*.app/Contents/Resources/julia/bin/julia` or `$HOME/.juliaup/bin/julia`
- **Linux**: `/usr/bin/julia` or `$HOME/.juliaup/bin/julia`

2.4 Must-have VS Code extensions

- **Julia** (julialang)
- **Quarto**
- **Rainbow CSV** (handy for data files)
- **GitLens** (git UX)

2.5 Quick check

Create `test.jl` and run in VS Code (Ctrl/Cmd+Enter):

```
println("Hello, Julia!")
x = [1, 2, 3, 4, 5]
println("Sum: ", sum(x))
```

2.6 Basic numerics

Now that your environment is set up, let's explore some basic Julia syntax.

```
1 + 2, sin(1.0)
```

```
(3, 0.8414709848078965)
```

2.6.1 Arithmetic

```
a = 10
b = 3
a + b, a - b, a * b, a / b
```

```
(13, 7, 30, 3.3333333333333335)
```

2.6.2 Arrays

```
x = [1, 2, 3, 4, 5]
sum(x), length(x)
```

```
(15, 5)
```

2.7 Functions

```julia
f(x) = x^2 + 2x + 1
f(3)
```

```
16
```

2.8 Control Flow

2.8.1 Conditionals

```julia
x = 5
if x > 0
    println("positive")
elseif x < 0
    println("negative")
else
    println("zero")
end
```

```
positive
```

2.8.2 Loops

```julia
for i in 1:5
    println(i^2)
end
```

```
1
4
9
16
25
```

2.9 Types and Structs

```julia
struct Point
    x::Float64
    y::Float64
end

p = Point(1.0, 2.0)
p.x, p.y
```

```
(1.0, 2.0)
```

3 Files & Data Manipulation

3.1 Reading and Writing Files

3.1.1 Text Files

```julia
# Writing to a file
open("example.txt", "w") do f
    write(f, "Hello, Julia!")
end

# Reading from a file
content = read("example.txt", String)
println(content)
```

```
Hello, Julia!
```

3.1.2 CSV Files

```julia
using CSV, DataFrames

# Read CSV
df = CSV.read("data.csv", DataFrame)

# Write CSV
CSV.write("output.csv", df)
```

3.2 DataFrames

DataFrames are tabular data structures similar to pandas in Python or data.frames in R.

3.2.1 Creating DataFrames

```julia
using DataFrames

# Create from columns
df = DataFrame(
    name = ["Alice", "Bob", "Charlie"],
    age = [25, 30, 35],
    city = ["Helsinki", "Espoo", "Tampere"]
)
```

	name	age	city
	String	Int64	String
1	Alice	25	Helsinki
2	Bob	30	Espoo
3	Charlie	35	Tampere

3.2.2 Basic Operations

```julia
# Select columns
df[:, :name]
```

```
3-element Vector{String}:
 "Alice"
 "Bob"
 "Charlie"
```

```julia
# Filter rows
filter(row -> row.age > 25, df)
```

	name	age	city
	String	Int64	String
1	Bob	30	Espoo
2	Charlie	35	Tampere

```julia
# Add new column
df.country = ["Finland", "Finland", "Finland"]
df
```

	name	age	city	country
	String	Int64	String	String
1	Alice	25	Helsinki	Finland
2	Bob	30	Espoo	Finland
3	Charlie	35	Tampere	Finland

3.2.3 Grouping and Aggregation

```
using Statistics

# Group by and summarize
gdf = groupby(df, :country)
combine(gdf, :age => mean => :avg_age)
```

	country	avg_age
	String	Float64
1	Finland	30.0

3.3 Working with Geoscientific Data

3.3.1 Loading Geophysical Data

```
using DelimitedFiles

# Read space-delimited data
data = readdlm("gravity_data.txt")

# Extract columns
x, y, gz = data[:, 1], data[:, 2], data[:, 3]
```

3.3.2 NetCDF Files

```
using NCDatasets

# Read NetCDF
ds = Dataset("climate_data.nc")
temp = ds["temperature"][:]
```

```
close(ds)
```

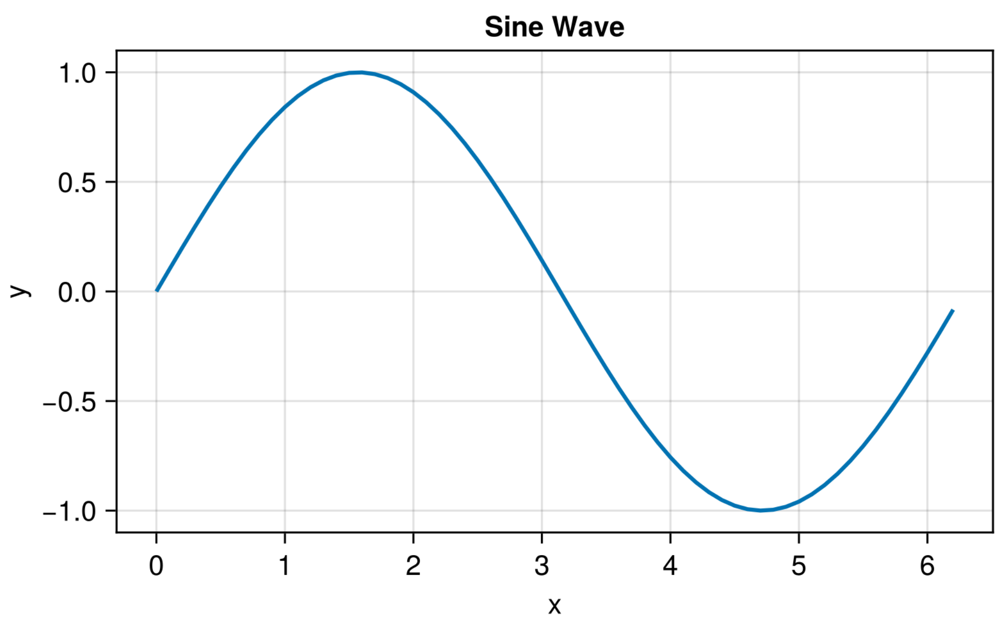
4 Plotting & Maps

4.1 Basic Plotting with CairoMakie

CairoMakie is a powerful plotting package that produces high-quality vector graphics and supports direct PDF output.

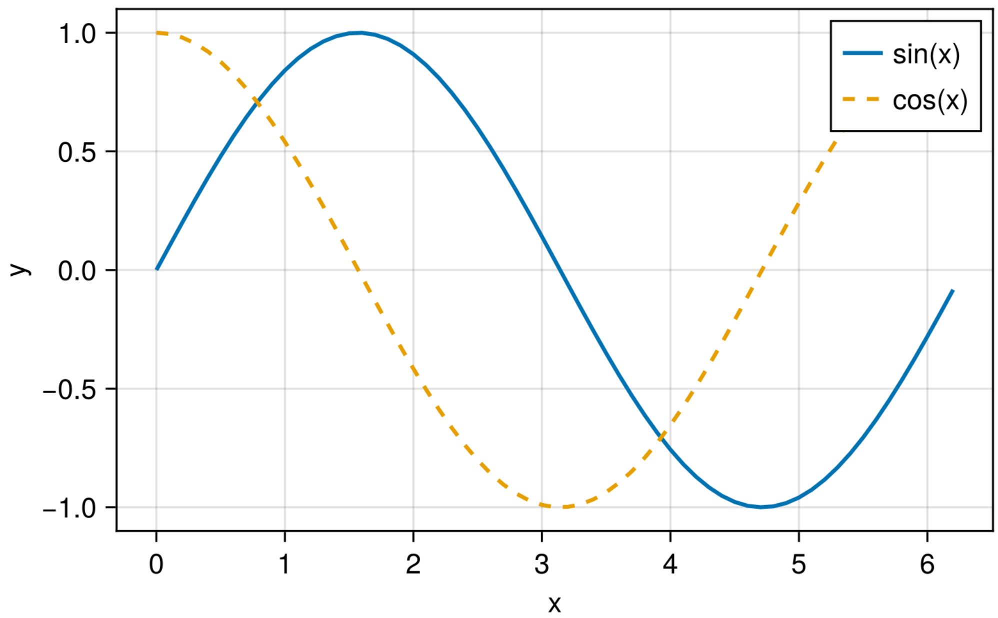
```
using CairoMakie
CairoMakie.activate!(type = "png")
```

```
x = 0:0.1:2π
y = sin.(x)
fig = lines(x, y, axis=(xlabel="x", ylabel="y", title="Sine Wave"),
            label="sin(x)", linewidth=2)
fig
```



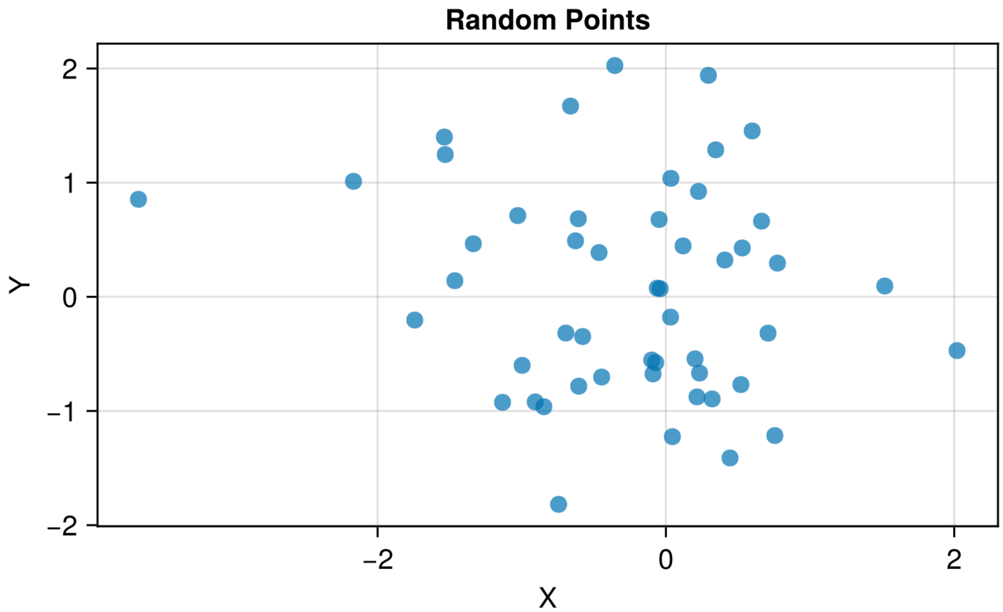
4.1.1 Multiple Series

```
fig = Figure()
ax = Axis(fig[1, 1], xlabel="x", ylabel="y")
lines!(ax, x, sin.(x), label="sin(x)", linewidth=2)
lines!(ax, x, cos.(x), label="cos(x)", linewidth=2, linestyle=:dash)
axislegend(ax)
fig
```



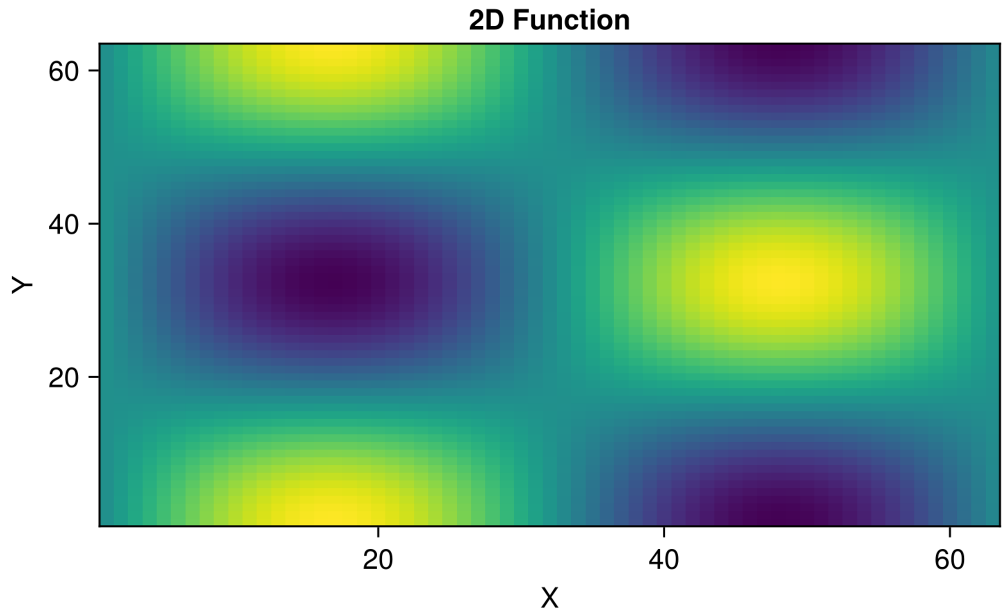
4.1.2 Scatter Plots

```
xs = randn(50)
ys = randn(50)
fig = scatter(xs, ys,
              axis=(xlabel="X", ylabel="Y", title="Random Points"),
              markersize=12, alpha=0.7)
fig
```



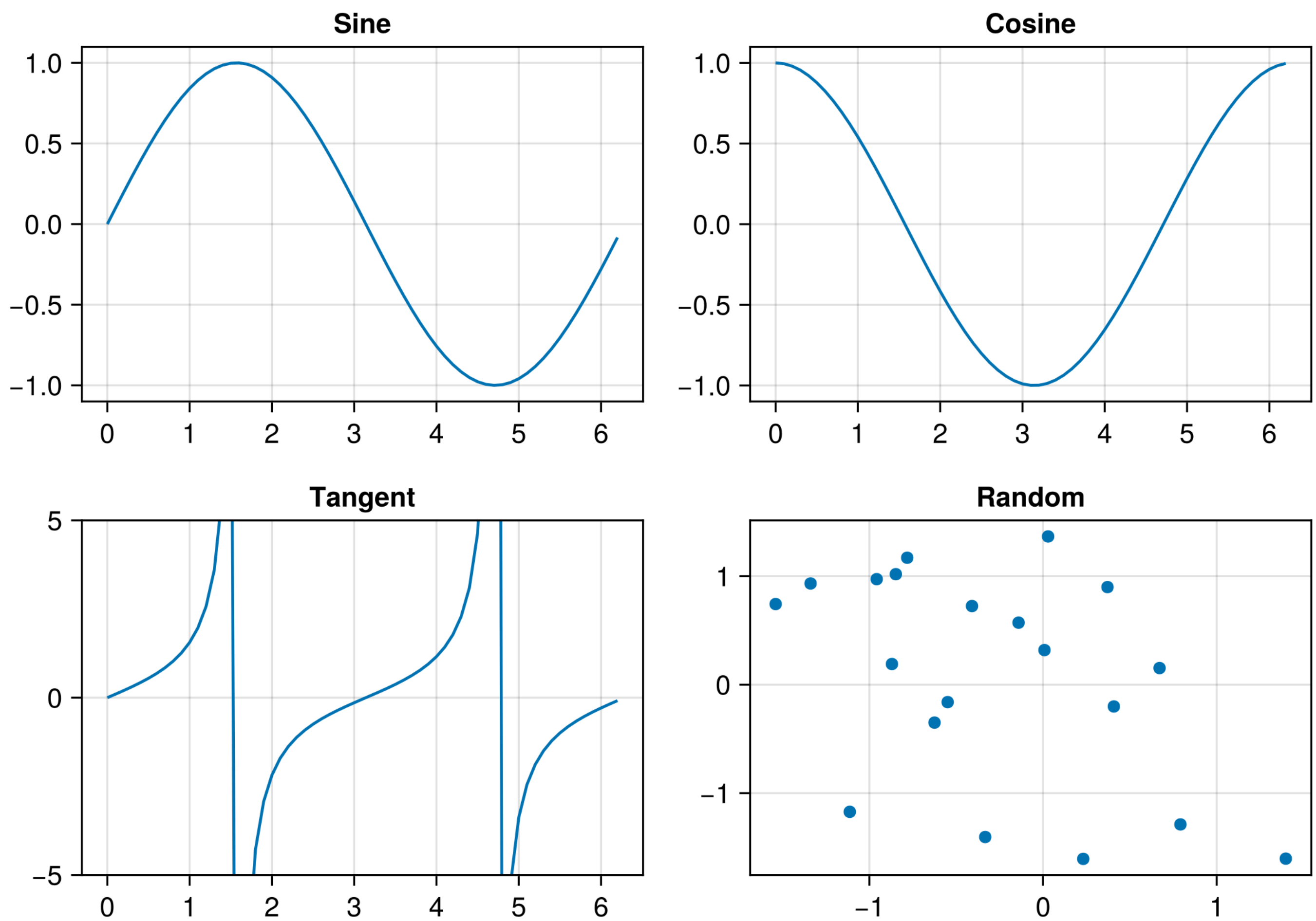
4.1.3 Heatmaps

```
z = [sin(xi) * cos(yi) for xi in 0:0.1:2π, yi in 0:0.1:2π]
fig = heatmap(z, axis=(xlabel="X", ylabel="Y", title="2D Function"),
              colormap=:viridis)
fig
```



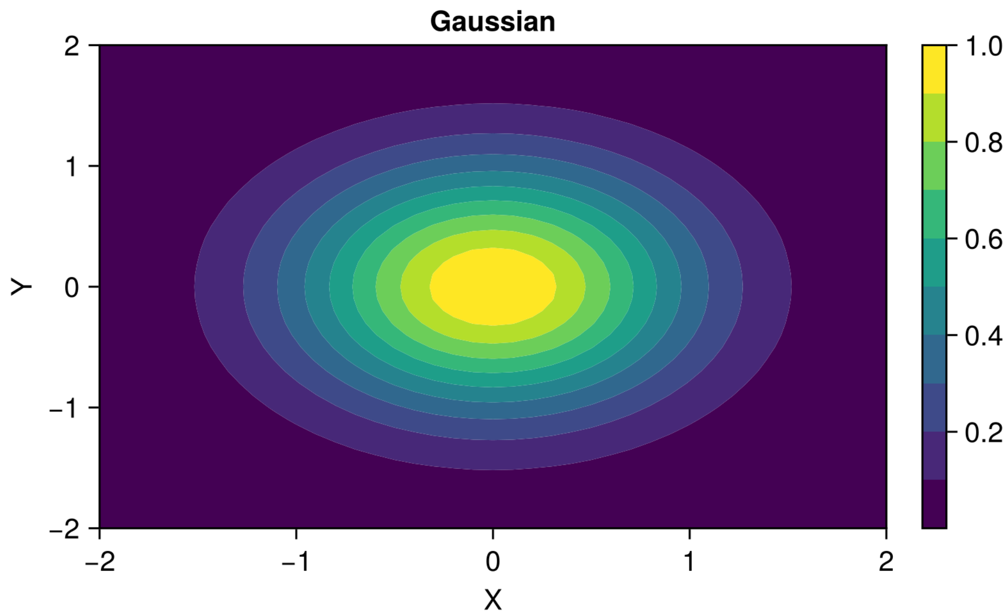
4.2 Subplots

```
fig = Figure(size=(700, 500))

ax1 = Axis(fig[1, 1], title="Sine")
lines!(ax1, x, sin.(x))

ax2 = Axis(fig[1, 2], title="Cosine")
lines!(ax2, x, cos.(x))

ax3 = Axis(fig[2, 1], title="Tangent", limits=(nothing, (-5, 5)))
lines!(ax3, x, tan.(x))

ax4 = Axis(fig[2, 2], title="Random")
scatter!(ax4, randn(20), randn(20))

fig
```



4.3 Contour Plots

```
x_cont = -2:0.1:2
y_cont = -2:0.1:2
z_cont = [exp(-(xi^2 + yi^2)) for xi in x_cont, yi in y_cont]

fig = Figure()
ax = Axis(fig[1, 1], xlabel="X", ylabel="Y", title="Gaussian")
co = contourf!(ax, x_cont, y_cont, z_cont, levels=10)
Colorbar(fig[1, 2], co)
fig
```



4.4 Geographic Maps

> ℹ Under Construction
>
> Geographic mapping examples with GeoMakie.jl will be added soon.

4.4.1 Coordinate Systems

```
# Example with projected coordinates
using CoordinateTransformations
using Proj4

# Transform from WGS84 to UTM
wgs84 = Proj4.Projection("+proj=longlat +datum=WGS84")
utm35 = Proj4.Projection("+proj=utm +zone=35 +datum=WGS84")
```

4.4.2 Plotting Geophysical Data

```julia
# Gravity anomaly map example
using CairoMakie

# Create synthetic gravity data
nx, ny = 50, 50
x_grav = range(0, 10, length=nx)
y_grav = range(0, 10, length=ny)
gz = [10 * exp(-((xi-5)^2 + (yi-5)^2)/2) for xi in x_grav, yi in y_grav]

fig = Figure()
ax = Axis(fig[1, 1],
          xlabel="Easting (km)",
          ylabel="Northing (km)",
          title="Bouguer Gravity Anomaly")
hm = heatmap!(ax, x_grav, y_grav, gz, colormap=:RdBu)
Colorbar(fig[1, 2], hm, label="mGal")
fig
```

5 Neural Networks

> **Under Construction**
>
> This chapter is currently being developed. Check back soon for updates!

5.1 Introduction to Neural Networks

> **Coming Soon**
>
> - Perceptrons and activation functions
> - Feedforward networks
> - Backpropagation

5.2 Flux.jl Basics

> **Coming Soon**
>
> - Installing Flux.jl
> - Building your first neural network
> - Training loops

5.3 Deep Learning Architectures

5.3.1 Convolutional Neural Networks (CNNs)

Coming Soon

- Convolution operations
- Pooling layers
- Image classification for geoscience

5.3.2 Recurrent Neural Networks (RNNs)

Coming Soon

- LSTM and GRU cells
- Time series prediction
- Sequence modeling for seismic data

5.3.3 Autoencoders

Coming Soon

- Encoder-decoder architecture
- Variational autoencoders
- Dimensionality reduction

5.4 Training Best Practices

Coming Soon

- Data preprocessing and normalization
- Batch training
- Learning rate scheduling
- Regularization techniques

5.5 GPU Computing with CUDA.jl

Coming Soon

- GPU setup
- Moving data to GPU
- Performance optimization

6 Inverse Modeling

> ⚠ Under Construction
>
> This chapter is currently being developed.

7 AI Surrogates

⚠ Under Construction

This chapter is currently being developed.

7.0.1 DeepONet

Coming Soon

- Branch and trunk networks
- Operator learning framework
- Geophysical applications

7.1 Gaussian Processes

Coming Soon

- GP fundamentals
- Kernel selection
- Scalable GPs

7.2 Reduced-Order Models

Coming Soon

- Proper Orthogonal Decomposition (POD)
- Dynamic Mode Decomposition (DMD)
- Neural network-enhanced ROM

7.3 Uncertainty Quantification

Coming Soon

- Epistemic vs aleatoric uncertainty
- Ensemble methods
- Bayesian neural networks

7.4 Multi-Fidelity Modeling

Coming Soon

- Combining cheap and expensive models
- Transfer learning
- Active learning for surrogates

8 Applications

⚠️ Under Construction

This chapter is currently being developed.